

Fundamentals of Data Science

Lecture 2 - Data wrangling

Adrian Ochs ¹ and Christian Roerig ²

Michaelmas 2026

¹Co-founder @ Quantum Leap, ex-QuantCo

²Data Scientist/ML Engineer @ QuantCo

Table of contents

1. GitHub

Merge conflicts

2. What is data wrangling?

How to decide on the right framework?

3. Tidy format

4. Processing variables

5. Merging tables

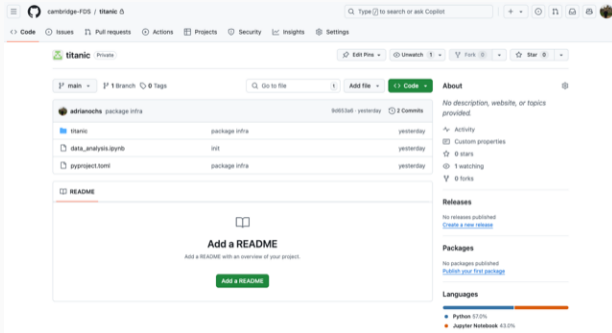
6. Handling null values - missing data

7. Save your data as Parquet file

8. Bonus

GitHub

GitHub let's you store your repo online and collaborate



You can push your changes to GitHub

Once you have saved your changes you can send them (i.e. "push") to GitHub:

```
1  git add .  
2  git commit -m "fix spelling mistake"  
3  git push  
4
```

And pull the changes others made

To update your repo locally with the changes that others made, "pull" them:

```
1  git pull
```

```
2
```

And pull the changes others made

To update your repo locally with the changes that others made, "pull" them:

```
1  git pull
```

Note: only pulls the changes for the branch you are on

Working locally and remotely with Git and GitHub

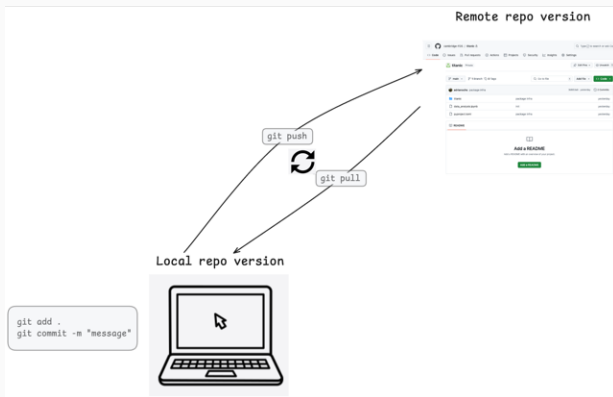


Figure 1: Icon from [here](#).

You can clone or fork the repositories of others

Create a copy of the repo to your own GitHub account (remote)

Clone

HTTPS SSH GitHub CLI

<https://github.com/jms/pandas-cookbook.git>

Clone using the web URL

Open with GitHub Desktop

Download ZIP

Create a local copy of the repository via:
git clone https://github...

Cloning for collaboration and forking for own copy

1. Cloning:

- 1.1 **"Downloads"** a local copy of a repository on your machine.
- 1.2 You can clone any repository that you have access to.
- 1.3 Typically used when you want to work on a project locally.
- 1.4 The cloned repository is directly linked to the original remote repository.
- 1.5 If you have write permissions, you can push changes directly to the original repo.

2. Forking:

- 2.1 Creates a **copy** of a repository in your GitHub (or similar platform) account.
- 2.2 It's a remote copy, not a local one.
- 2.3 Typically used when you want to contribute to a project you don't have write access to.
- 2.4 The forked repo is your own copy, separate from the original.
- 2.5 You can make changes to your fork without affecting the original repo.
- 2.6 Often used as a starting point for pull requests in open-source contributions.

Pull requests to make a change to your code base

The screenshot shows a GitHub pull request page for the repository 'cambridge-FDS / fun_ds'. The title of the pull request is '[WIP] simple data retrieval example #1'. It was opened by ChristianRoerig, who wants to merge 1 commit into the 'main' branch from the 'data_retrieval' branch. The interface includes tabs for Code, Issues, Pull requests (1), Actions, Projects, Security, Insights, and Settings. Below the title, there are statistics: Conversation (0), Commits (1), Checks (0), and Files changed (3). A comment by ChristianRoerig, made 3 weeks ago, contains a checklist with one item: 'Added a CHANGELOG.rst entry'. Below the comment, there is a link to the pull request. A green box highlights the 'Require approval from specific reviewers before merging' rule, which ensures specific people approve pull requests before they're merged. Another green box indicates that the branch has no conflicts with the base branch, allowing for automatic merging. At the bottom, there is a 'Merge pull request' button and a link to open the pull request in GitHub Desktop or view command line instructions. The 'Add a comment' section is visible at the bottom of the page.

cambridge-FDS / fun_ds

Type to search or ask Copilot

< Code Issues Pull requests 1 Actions Projects Security Insights Settings

[WIP] simple data retrieval example #1

Open ChristianRoerig wants to merge 1 commit into main from data_retrieval

Conversation 0 Commits 1 Checks 0 Files changed 3

ChristianRoerig commented 3 weeks ago

Checklist

- ☐ Added a CHANGELOG.rst entry

simple data retrieval example f2fc375

Require approval from specific reviewers before merging Add rule X

Rulesets ensure specific people approve pull requests before they're merged.

This branch has no conflicts with the base branch

Merging can be performed automatically.

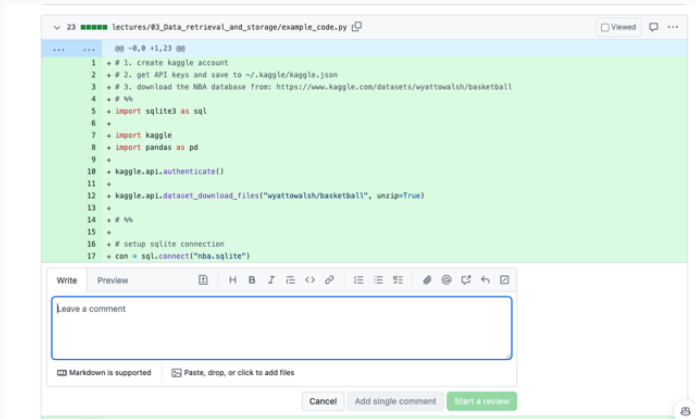
Merge pull request You can also open this in GitHub Desktop or view command line instructions.

Add a comment

Write Preview

Add your comment here...

PR reviews let you check the code of your co-workers



PR review etiquette



- Don't be personal: talk about the code.
 - "A square bracket is missing" vs. "You forgot a square bracket"
- Also: no one is criticising you personally. It is just about the code.
- No context: be extra nice since context might get lost in writing
 - a short comment might be added because you were in a rush but someone perceives it as rude
- Compliment others when there was a particularly good solution.
 - Choose nice emojis!
- This is about finding mistakes. Finding mistakes is a good thing!
- Everyone makes mistakes → that's why PR reviews exist!

Use Git and GitHub for your problems set

For each problem set you will work in groups. When indicated, you work with the repos that we provide on the GitHub page for this course. A team member has to:

- Go to <https://github.com/orgs/cambridge-FDS/repositories>
- Click on the repo for the problem set, e.g. "titanic"
- Fork the repo

Then, all team members:

- Clone the forked repo from that team members' GitHub
- Allocate tasks for each other
- Create a branch for your task
- Create a Pull Request (PR)
- Work on your task and commit and push frequently!
- Once you are done, ask for a PR review
- When everyone agrees, merge the branch into main

One team member submits an email with the links to the repositories.

Showcase the Git and GitHub workflow

Live session:

- Create a repository
- Clone the repository
- Create a branch
- Make a change and push to GitHub
- Open a Pull Request
- Review the PR
- Merge the PR to main

Merge conflicts

Collaboration can lead to merge conflicts

- Collaboration leads to merge conflicts

Collaboration can lead to merge conflicts

- Collaboration leads to merge conflicts
 - Someone else merged something into main at the same place as you

Collaboration can lead to merge conflicts

- Collaboration leads to merge conflicts
 - Someone else merged something into main at the same place as you
- Even without a merge conflict, you might have broken the code!

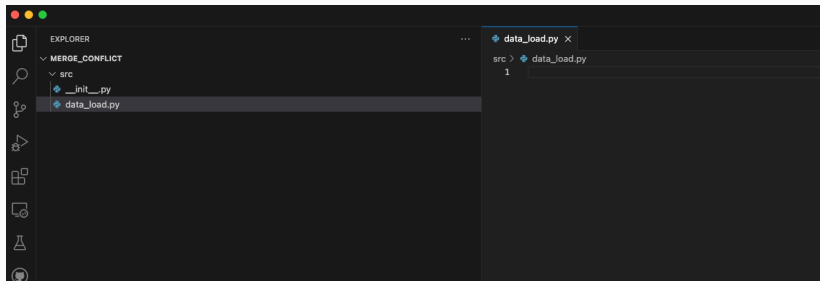
Collaboration can lead to merge conflicts

- Collaboration leads to merge conflicts
 - Someone else merged something into main at the same place as you
- Even without a merge conflict, you might have broken the code!
- This is why it is good practice to have:
 - Unit tests: small test that make sure all functions work as intended (covered in Research Computing)
 - Integration tests: tests that make sure that your training pipeline runs through correctly (feed in some data and check that the final result fits expectations)
 - Smoke tests: just making sure that something comes out at the end of your pipeline (i.e. there is no break that stops the pipeline)

Collaboration can lead to merge conflicts

- Collaboration leads to merge conflicts
 - Someone else merged something into main at the same place as you
- Even without a merge conflict, you might have broken the code!
- This is why it is good practice to have:
 - Unit tests: small test that make sure all functions work as intended (covered in Research Computing)
 - Integration tests: tests that make sure that your training pipeline runs through correctly (feed in some data and check that the final result fits expectations)
 - Smoke tests: just making sure that something comes out at the end of your pipeline (i.e. there is no break that stops the pipeline)
- Here, just make sure after merging that your code on the main branch still runs through.

Let's look at how we handle a merge conflict



Create a new branch and add a data load function

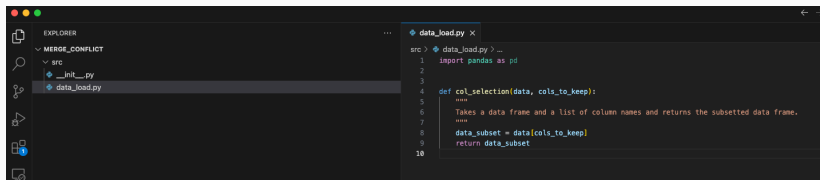
The screenshot shows the Visual Studio Code interface. On the left, the Explorer pane shows a project structure with a 'src' directory containing 'data_load.py' (marked with a blue diamond icon). The main editor area shows the 'data_load.py' file with the following Python code:

```
src > data_load.py > ...
1 import pandas as pd
2
3
4 def data_loader(path):
5     """
6     This function takes in a path and loads the .csv file using the separator ';'.
7     """
8     df = pd.read_csv(path, sep=";")
9     return df
10
```

At the bottom, the Output pane shows the following terminal output:

```
(base) ➔ merge_conflict git:(main) git checkout -b data_load_fct
Switched to a new branch 'data_load_fct'
(base) ➔ merge_conflict git:(data_load_fct) []
```

Create another branch from main to create the column selector function



The screenshot shows a code editor interface with a dark theme. On the left, the 'EXPLORER' sidebar displays a file tree with a 'MERGE_CONFLICT' section containing files `__init__.py` and `data_load.py`. The `data_load.py` file is selected. The main editor area shows the content of `data_load.py`, which includes an import statement and a function definition. The file name `data_load.py` is shown in the tab at the top of the editor.

```
src > data_load.py > ...
1  import pandas as pd
2
3
4  def col_selection(data, cols_to_keep):
5      """
6      Takes a data frame and a list of column names and returns the subsetted data frame.
7      """
8      data_subset = data[cols_to_keep]
9      return data_subset
10
```


We create a PR for both functions

 **merge_conflict** Private

 Edit Pins  Unwatch 1

 **col_selector** had recent pushes 1 minute ago [Compare & pull request](#)

 main  2 Branches  0 Tags  Add file  Code

 **adrianochs** Init 97f5928 · 47 minutes ago 🕒 2 Commits

 **src** init 47 minutes ago

 **README**



Add a README

Add a README with an overview of your project.

[Add a README](#)

Merging the PR for the data loader

Feature: add a function to load the data #2

Merged adrianochs merged 1 commit into `main` from `data_load_fct` 9 minutes ago

Conversation 0 Commits 1 Checks 0 Files changed 1

adrianochs commented 10 hours ago

Add a function to load the data

add data_loader function 7a1ab27

adrianochs self-assigned this 10 hours ago

adrianochs merged commit 6e716e3 into `main` 9 minutes ago Revert

Add a comment

Write Preview

⌨ H B I

Add your comment here

And now there is a merge conflict with the column selector PR

Feature: add a function to subset a data frame #1

 **Open** adrianochs wants to merge 1 commit into `main` from `col_selector` 

Conversation 0 Commits 1 Checks 0 Files changed 1

 **adrianochs** commented 10 hours ago ...

This PR creates a function to subset a data frame with a list of columns.



  add col selection function cdfbec3

 **adrianochs** self-assigned this 10 hours ago

  **This branch has conflicts that must be resolved** Resolve conflicts

Use the [web editor](#) or the [command line](#) to resolve conflicts.

Conflicting files

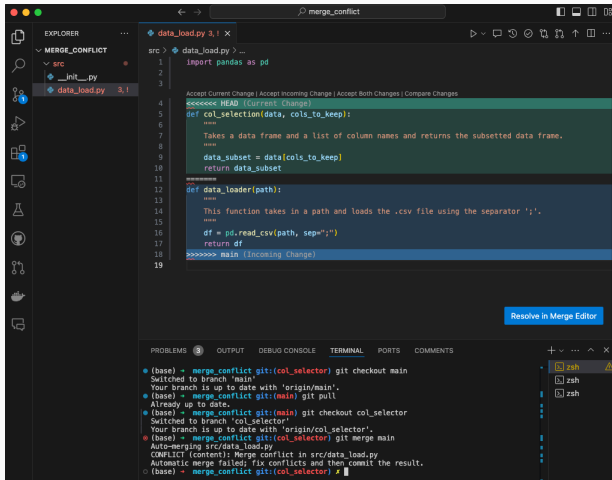
`src/data_load.py`

Merge pull request  You can also [open this in GitHub Desktop](#) or view [command line instructions](#).

Steps to resolve the merge conflict

1. Locally, checkout your main branch
2. Pull all changes on main
3. Checkout the branch with the merge conflict
4. Merge main into the branch
5. Resolve the merge conflict in VS code
6. Add, commit, and push your changes
7. Check on GitHub that you now have no merge conflict anymore

A merge conflict in VS Code

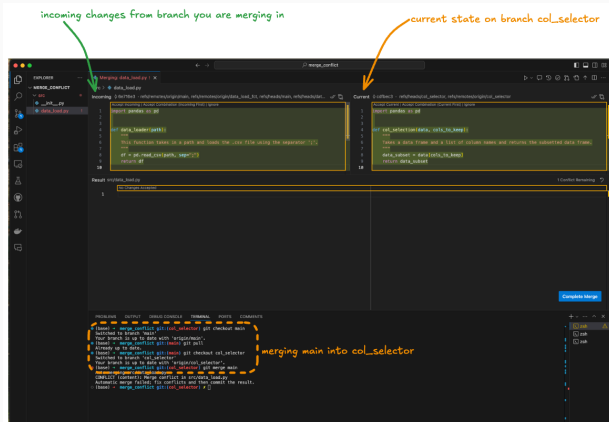


```
src > data_load.py > ...
1 import pandas as pd
2
3
4 <<<<<< HEAD (Current Change)
5 def col_selection(data, cols_to_keep):
6     """
7     Takes a data frame and a list of column names and returns the subsetted data frame.
8     """
9     data_subset = data[cols_to_keep]
10    return data_subset
11
12 >>>>>> main (Incoming Change)
13 def data_loader(path):
14     """
15     This function takes in a path and loads the .csv file using the separator ';'.
16     """
17     df = pd.read_csv(path, sep=";")
18     return df
19
```

Resolve in Merge Editor

```
(base) ➜ merge_conflict git:(col_selector) git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
(base) ➜ merge_conflict git:(main) git pull
Already up to date.
(base) ➜ merge_conflict git:(main) git checkout col_selector
Switched to branch 'col_selector'
Your branch is up to date with 'origin/col_selector'.
(base) ➜ merge_conflict git:(col_selector) git merge main
Auto-merging src/data_load.py
CONFLICT (content): Merge conflict in src/data_load.py
Automatic merge failed; fix conflicts and then commit the result.
(base) ➜ merge_conflict git:(col_selector) *
```

Switching to the merge editor



Deciding which parts to keep

Merging: data_load.py | X

src > data_load.py

Incoming 6e716e3 · refs/remotes/origin/main, refs/remotes/origin/data_load... ✓

```
1 import pandas as pd
2
3
4 def data_loader(path):
5     """
6     This function takes in a path and loads the .csv file using
7
8     df = pd.read_csv(path, sep=";")
9     return df
10
```

Current cdfbec3 · refs/heads/col_selector, refs/remotes/origin/col_selector ✓

```
1 import pandas as pd
2
3
4 def col_selection(data, cols_to_keep):
5     """
6     Takes a data frame and a list of column names and returns the
7
8     data_subset = data[cols_to_keep]
9     return data_subset
10
```

Result src\data_load.py 0 Conflicts Remaining ?

1	No Changes Accepted
---	---------------------

Resolving the merge conflict

The screenshot shows a code editor interface for resolving a merge conflict. At the top, the title bar says "Merging: data_load.py | X". Below it, the "src" directory is selected, and the file "data_load.py" is open, showing the "col_selection" function. The "Incoming" branch (0 6e716e3) is selected, and the "Current" branch (0 cdfbec3) is also selected. The "Incoming" code shows a "data_loader" function that reads a CSV file. The "Current" code shows a "col_selection" function that filters a DataFrame by columns. The "Result" view shows the merged code, which includes both functions. A "Complete Merge" button is highlighted with an orange arrow.

```
import pandas as pd

def data_loader(path):
    """
    This function takes in a path and loads the .csv file using
    """
    df = pd.read_csv(path, sep=";")
    return df

def col_selection(data, cols_to_keep):
    """
    Takes a data frame and a list of column names and returns the
    """
    data_subset = data[cols_to_keep]
    return data_subset
```

- Use "git commit" in the terminal to finish the merge and "git push".

Everything is green again on GitHub and we can merge

Feature: add a function to subset a data frame #1

Open adrianochs wants to merge 2 commits into `main` from `col_selector`

Conversation 0 Commits 2 Checks 0 Files changed 1

adrianochs commented 11 hours ago

This PR creates a function to subset a data frame with a list of columns.

add col selection function cdfbec3

adrianochs self-assigned this 11 hours ago

Merge branch 'main' into col_selector 86478a4

Require approval from specific reviewers before merging
[Rulesets](#) ensure specific people approve pull requests before they're merged. Add rule ×

Continuous integration has not been set up
[GitHub Actions](#) and [several other apps](#) can be used to automatically catch bugs and enforce style.

This branch has no conflicts with the base branch
Merging can be performed automatically.

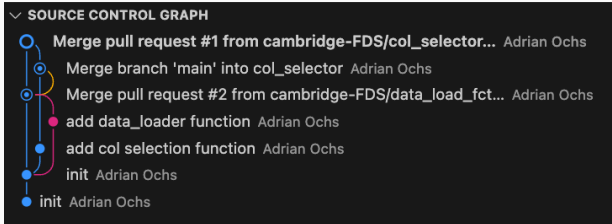
Merge pull request You can also [open this in GitHub Desktop](#) or view [command line instructions](#).

Add a comment

Write Preview

- After your press merge, do not forget to "git pull" on main to stay up to date.

A quick look at the Git branch



Quick help: Git config

- When you install Git, you will have to configure your username and email address.
- To ensure that commits are attributed to you, use an email address that is connected to your account on GitHub.
- See a good explanation on [GitHub](#)

```
1 git config --global user.name "YOUR_NAME"  
2 git config --global user.email "YOUR_EMAIL"  
3
```

To open your global git config file:

```
1 git config --global --edit  
2
```

What is data wrangling?

The data stage



Figure 2: Data science pipeline

What is data wrangling?

Data wrangling is the process of converting raw data into a *usable* form by:

- Reshaping or grouping your data (tidy format)

What is data wrangling?

Data wrangling is the process of converting raw data into a *usable* form by:

- Reshaping or grouping your data (tidy format)
- Processing strings
 - e.g. deleting punctuation: "hello!?" → "hello"

What is data wrangling?

Data wrangling is the process of converting raw data into a *usable* form by:

- Reshaping or grouping your data (tidy format)
- Processing strings
 - e.g. deleting punctuation: "hello!?" → "hello"
- Processing dates
 - e.g. extracting the year from "2023-10-01" → 2023

What is data wrangling?

Data wrangling is the process of converting raw data into a *usable* form by:

- Reshaping or grouping your data (tidy format)
- Processing strings
 - e.g. deleting punctuation: "hello!?" → "hello"
- Processing dates
 - e.g. extracting the year from "2023-10-01" → 2023
- Sub-setting with comparison operation (binning)
 - e.g. "age_group = "underage" if age < 18 else "adult""

What is data wrangling?

Data wrangling is the process of converting raw data into a *usable* form by:

- Reshaping or grouping your data (tidy format)
- Processing strings
 - e.g. deleting punctuation: "hello!?" → "hello"
- Processing dates
 - e.g. extracting the year from "2023-10-01" → 2023
- Sub-setting with comparison operation (binning)
 - e.g. "age_group = "underage" if age < 18 else "adult""
- Handling null values (i.e. missing values)

What is data wrangling?

Data wrangling is the process of converting raw data into a *usable* form by:

- Reshaping or grouping your data (tidy format)
- Processing strings
 - e.g. deleting punctuation: "hello!?" → "hello"
- Processing dates
 - e.g. extracting the year from "2023-10-01" → 2023
- Sub-setting with comparison operation (binning)
 - e.g. "age_group = "underage" if age < 18 else "adult""
- Handling null values (i.e. missing values)
- Handling outliers
- Standardising or normalising your data

What is data wrangling?

Data wrangling is the process of converting raw data into a *usable* form by:

- Reshaping or grouping your data (tidy format)
- Processing strings
 - e.g. deleting punctuation: "hello!?" → "hello"
- Processing dates
 - e.g. extracting the year from "2023-10-01" → 2023
- Sub-setting with comparison operation (binning)
 - e.g. "age_group = "underage" if age < 18 else "adult""
- Handling null values (i.e. missing values)
- Handling outliers
- Standardising or normalising your data
- Bringing multiple tables together (merging)

Thinking ahead when wrangling data

- Some steps depend on your modelling approach (e.g. standardisation useful for neural networks but not for Random Forests)

Thinking ahead when wrangling data

- Some steps depend on your modelling approach (e.g. standardisation useful for neural networks but not for Random Forests)
- But also on what and when you want to predict

Thinking ahead when wrangling data

- Some steps depend on your modelling approach (e.g. standardisation useful for neural networks but not for Random Forests)
- But also on what and when you want to predict
- Thus, when you wrangle your data you have to think about your modelling approach

Thinking ahead when wrangling data

- Some steps depend on your modelling approach (e.g. standardisation useful for neural networks but not for Random Forests)
- But also on what and when you want to predict
- Thus, when you wrangle your data you have to think about your modelling approach
- "Knowing how to properly clean and assemble your data will set you miles apart from others in your field." (R. Dimond (2024) "Data Wrangling in Python")

Data wrangling is learned hands-on

Idea for this lecture:

- Data wrangling is best learned hands-on
- A lecture is inferior to just doing it
- Your problem set will provide hands-on practice
- Here, we collected
 - things that come up often
 - things that often go wrong (but you do not think about)
 - "case studies" on:
 - how to decide on switching frameworks?
 - data leakage during data wrangling
 - how to feed your feature into the model?

How to decide on the right
framework?

Polars vs. Pandas for data wrangling

- For a long time, Pandas was the go to library in Python to deal with data.
- Recently, a new library, Polars, has gained a lot of popularity.
- It is faster (written in Rust), allows for lazy data frames (see [tutorial](#)), and has a very nice syntax.

When to switch libraries?

What were the reasons for switching libraries and how to convince your colleagues:

- trust: GitHub stars
- active development: check who and how many people are contributing
- show the benefit: speed comparisons
- other:
 - syntax clearer
 - allowed to deal with large data sets and hence, we could do unit test on Python code; harder with SQL.

When to switch libraries?

What were the reasons for switching libraries and how to convince your colleagues:

- trust: GitHub stars
- active development: check who and how many people are contributing
- show the benefit: speed comparisons
- other:
 - syntax clearer
 - allowed to deal with large data sets and hence, we could do unit test on Python code; harder with SQL.

→ Do not get married to a library. Things change!

→ Refactoring code is something that you will do all the time. In your exercises, you will have to rewrite Pandas code in Polars.

How to stay up to date and learn new things like Polars

Stay up-to-date:

- Twitter
- Medium: [Towards Data Science](#)
- Newsletters: [Data Science Weekly](#), [The Pragmatic Engineer](#)

Learn:

- Beginner: [DataCamp](#), [Code Academy](#)
- SWE: [calmcode](#)
- Intermediate: [Kaggle competitions](#)
- Advanced: Repos, tutorials, coding things yourself, academic papers (rare)

First steps - Let's tidy up

Tidy format - a usable form

Tidy format means:

- Each column corresponds to one variable in the data set.
- Each row corresponds to one observation.
- → Target variable and feature set are at the same observational level!

The opposite - a messy data set

On the other hand, a messy data set might have:

- Values in the column name
- Multiple variables stored in one column
- Variables stored in both rows and columns
- Multiple rows corresponding to one observation

Example - a messy data set

What is the problem?

	country	1960	1961	1962	1963
0	Germany	2.41	2.44	2.47	2.49
1	South Korea	6.16	5.99	5.79	5.57

Example - a messy data set

What is the problem?

	country	1960	1961	1962	1963
0	Germany	2.41	2.44	2.47	2.49
1	South Korea	6.16	5.99	5.79	5.57

- There are values in the column name
 - Hard to plot, e.g., fertility rates over time
- Table in wide format; we don't have one observation per row
 - We want country per year

Use pandas melt to go from wide to long

```
pd.melt(df, id_vars=['country'], var_name='year', value_name='fertility')
```

var_name: gives name to the column that contains the former column names.

	country	1960	1961	1962	1963
0	Germany	2.41	2.44	2.47	2.49
1	South Korea	6.16	5.99	5.79	5.57

id_vars: specify columns that should be kept as is.

value_name: gives name to the column that contains the values from the melted columns.

Use pandas melt to go from wide to long

	country	1960	1961	1962	1963
0	Germany	2.41	2.44	2.47	2.49
1	South Korea	6.16	5.99	5.79	5.57

`pd.melt(...)`

	country	year	fertility
0	Germany	1960	2.41
1	Germany	1961	2.44
2	Germany	1962	2.47
3	Germany	1963	2.49
4	South Korea	1960	6.16
5	South Korea	1961	5.99
6	South Korea	1962	5.79
7	South Korea	1963	5.57

Why is it called melt?

Courtesy of Claude (Anthropic's GPT):

- **Reshaping:** Melting ice cream loses its original shape and spreads out. Similarly, melting data spreads out the information contained in multiple columns into more rows.

Example - messy data?

What's the problem?

	household_id	person	age	income	female
0	37	1	20	10000	False
1	37	2	19	5300	True
2	37	3	19	4700	False
3	241	1	50	90000	True
4	242	1	29	20000	False
5	155789	1	58	5000	False
6	155789	2	61	110000	True
7	155789	3	15	NaN	False

Example - messy data?

What's the problem?

	household_id	person	age	income	female
0	37	1	20	10000	False
1	37	2	19	5300	True
2	37	3	19	4700	False
3	241	1	50	90000	True
4	242	1	29	20000	False
5	155789	1	58	5000	False
6	155789	2	61	110000	True
7	155789	3	15	NaN	False

It depends:

- If we want to work on the individual level, it's fine
- If we want to work on household level, then we do not have one observation per row

Example - bring to household level with creativity

	household_id	person	age	income	female
0	37	1	20	10000	False
1	37	2	19	5300	True
2	37	3	19	4700	False
3	241	1	50	90000	True
4	242	1	29	20000	False
5	155789	1	58	5000	False
6	155789	2	61	110000	True
7	155789	3	15	NaN	False



	household_id	size_hh	mean_age	min_age	max_age	nr_children	nr_female	mean_income	total_income	main_earner_female
0	37	3	19.3	19	20	0	1	6666.6	20000	no
1	241	1	50	50	50	0	1	90000	90000	yes
2	242	1	29	29	29	0	0	20000	20000	no
3	155789	3	944.7	15	61	1	1	38000	115000	yes

- You lose some information when aggregating data
- Being creative helps you come up with features to summarise the data

Case study: How to feed your features into the model

Having to manipulate a feature to be able to feed it into the model happens all the time:

- Case study I: predict length of hospital stay based on (among others) patient's diagnosis
- What's the problem?

Case study: How to feed your features into the model

Having to manipulate a feature to be able to feed it into the model happens all the time:

- Case study I: predict length of hospital stay based on (among others) patient's diagnosis
- What's the problem?
 - Medical diagnoses are defined in so-called ICD codes
 - There are almost 70,000 ICD codes
 - How do you feed that into your model?
 - Can your model handle ("string") categorical variables?

Aside: one-hot encoding explained

Before one-hot encoding:

Color

Red

Green

Blue

Green

Red

After one-hot encoding:

Red	Green	Blue
1	0	0
0	1	0
0	0	1
0	1	0
1	0	0

Example - bring to household level with creativity

	household_id	person	age	income	female
0	37	1	20	10000	False
1	37	2	19	5300	True
2	37	3	19	4700	False
3	241	1	50	90000	True
4	242	1	29	20000	False
5	155789	1	58	5000	False
6	155789	2	61	110000	True
7	155789	3	15	NaN	False



	household_id	size_hh	mean_age	min_age	max_age	nr_children	nr_female	mean_income	total_income	main_earner_female
0	37	3	19.3	19	20	0	1	6666.6	20000	no
1	241	1	50	50	50	0	1	90000	90000	yes
2	242	1	29	29	29	0	0	20000	20000	no
3	155789	3	944.7	15	61	1	1	38000	115000	yes

How to create your toy data frame

```
1  import pandas as pd
2
3  df = pd.DataFrame({
4      'household_id': [37, 37, 37, 241, ...],
5      ...
6      'female': [False, True, False, True, ...]
7  })
8
```

- Always a good idea to try your code on a small toy data frame that covers the complexity you need but is easy to handle
- This is what you will often do for so-called unit tests (covered in different course)

Have a test data frame on your side when you wrangle

Does Polars overwrites null values with False when you do a string comparison?

```
1     test = pl.DataFrame({"dash": ["29616-0759", None, "83"  
    ]})  
2     test.with_columns(rows_with_dashes=pl.col("dash").str.  
    contains("-"))  
3
```

Illustration of working with a test data frame

```
[2] import polars as pl
✓ 0.0s
```

```
[3] test = pl.DataFrame({"dash": ["29616-0759", None, "83"]})
test
✓ 0.0s
```

```
... shape: (3, 1)
  dash
  str
"29616-0759"
  null
  "83"
```

```
[4] test.with_columns(rows_with_dashes=pl.col("dash").str.contains("-"))
✓ 0.0s
```

```
... shape: (3, 2)
  dash  rows_with_dashes
  str          bool
"29616-0759"      true
  null           null
  "83"           false
```


How to aggregate with Pandas' groupby

```
1  import pandas as pd
2
3  # Aggregate to household level
4  household_df = df.groupby('household_id').agg({
5      'person': 'count',
6      'age': 'mean',
7      'income': 'mean',
8      'female': 'sum'
9  })
10
11 # Rename columns
12 household_df.columns = ['household_size', 'mean_age', '
13 mean_income', 'num_female']
14
15 # Reset the index to make household_id a column
16 household_df = household_df.reset_index()
```

We can also use polars - it's not too different

```
1  import polars as pl
2
3  household_df = (
4  df.groupby("household_id")
5  .agg([
6      pl.count("person").alias("household_size"),
7      pl.mean("age").alias("mean_age"),
8      pl.mean("income").alias("mean_income"),
9      pl.sum("female").alias("num_female")
10 ])
11 )
12
```

How would you create the 'main_earner_female' column

	household_id	person	age	income	female
0	37	1	20	10000	False
1	37	2	19	5300	True
2	37	3	19	4700	False
3	241	1	50	90000	True
4	242	1	29	20000	False
5	155789	1	58	5000	False
6	155789	2	61	110000	True
7	155789	3	15	NaN	False

	household_id	size_hh	mean_age	min_age	max_age	nr_children	nr_female	mean_income	total_income	main_earner_female
0	37	3	19.3	19	20	0	1	6666.6	20000	no
1	241	1	50	50	50	0	1	90000	90000	yes
2	242	1	29	29	29	0	0	20000	20000	no
3	155789	3	944.7	15	61	1	1	38000	115000	yes

```
1  # Define a function to check if the highest earner is female
2  def highest_earner_female(group):
3      max_income_row = group.loc[group['income'].idxmax()]
4      return max_income_row['female']
5
6  # Apply that function during the groupby
7  df.groupby('household_id').apply(highest_earner_female)
```

In polars

```
1  import polars as pl
2
3  # Filtering allows you to find the highest earner row
  directly
4  household_df = df.group_by('household_id').agg([
5      pl.col('female').filter(pl.col('income') == pl.col('
  income')).max().alias('highest_earner_female')
6  ]).sort('household_id')
7
```

First steps - Processing variables

Renaming your variables

- Not always a good idea: when you work for a client, they are used to the variable names, so better to keep them
- But sometimes it is very helpful, if the variables have no (obvious) meaning
- **Tip:** Keep a (Python) dictionary of the renaming you have done for communication purposes and easy re-mapping of variable names

How to rename your columns

```
1  rename_cols = {  
2      "old_column_name1": "new_column_name1",  
3      "old_column_name2": "new_column_name2",  
4  }  
5  
6  # Rename the columns  
7  df = df.rename(columns=rename_cols)
```


Good practice: write your hard coded variables at the top

```
1  import pandas as pd
2
3  DATA_PATH = "data/"
4  FAKE_PATH = DATA_PATH + "fake_data.csv"
5  RENAME_COLS = {"col_1": "strings", "col_2": "integers", "col_3": "floats"}
6
7
8  def create_simple_dataframe():
9      data = {
10         "col_1": ["A", "B", "C", "D", "E"],
11         "col_2": [1, 2, 3, 4, 5],
12         "col_3": [10.5, 20.5, 30.5, 40.5, 50.5],
13     }
14     df = pd.DataFrame(data)
15     return df
16
17
18 df = create_simple_dataframe()
19 df = df.rename(RENAME_COLS)
20 df.to_csv(FAKE_PATH)
21
```

Processing strings

Typical string operations:

- All lower case
- Length (number of words, number of characters)
- Regex (how many times does the word "econ" occur; sentiment analysis)
 - Delete punctuation
 - Replace all numbers
 - Count occurrence of special words (e.g. sentiment analysis)

Look up python string methods [here](#) and Polars [here](#).

String to lower and counting words

```
1 sentence = "The Quick Brown Fox Jumps Over The Lazy Dog"
2
3 lowercase_sentence = sentence.lower()
4
5 word_count = len(lowercase_sentence.split())
6
7 print(f"Original sentence: {sentence}")
8 print(f"Lowercase sentence: {lowercase_sentence}")
9 print(f"Word count: {word_count}")
10
11 # Output: Original sentence: The Quick Brown Fox Jumps Over
12 #          The Lazy Dog
13 # Output: Lowercase sentence: the quick brown fox jumps over
14 #          the lazy dog
15 # Output: Word count: 9
```

Regex (regular expressions)

Regular expressions are used for pattern matching and text manipulation in python.

For example, you can match to extract, delete or replace using:

- Literal characters: 'a' matches 'a'
- Special characters:
 - `.` (dot): matches any character except newline
 - ...
- Character classes:
 - `\d`: matches any digit (0-9)
 - `\w`: matches any alphanumeric character (a-z, A-Z, 0-9, _)
 - ...

A regex example

```
1  import re
2
3  sentence = "this. is a..test!"
4  cleaned = re.sub(r'^\w\s', '', sentence)
5  print(cleaned)
6  # Output: "this is atest"
7
```

- `r`: Raw string literal in Python, which treats backslashes as literal characters.
- `[]`: Square brackets in regex define a character set. It matches any single character that is inside the brackets.
- `^`: Means "match any character that is NOT in this set"
- `\w`: This is a shorthand character class that matches any word character
- `\s`: Matches white space character (spaces, tabs, newlines)

A regex example

So, `r'[\w\s]'` means:

- Match any single character that is not:
 - A word character
 - Or a white space character

And `re.sub(r'[\w\s]', "", sentence)` means

- Replace all matches (i.e. all punctuation and special characters) with an empty string

Sentiment analysis: a minimal example

```
1  import re
2
3  def analyze_sentiment(text):
4      positive_pattern = r'\b(good|great|excellent)\b'
5      negative_pattern = r'\b(bad|terrible|awful)\b'
6
7      # Convert text to lowercase for case-insensitive
      matching
8      text = text.lower()
9
10     positive_count = len(re.findall(positive_pattern, text))
11     negative_count = len(re.findall(negative_pattern, text))
12
13     if positive_count > negative_count:
14         sentiment = "Positive"
15     elif negative_count > positive_count:
16         sentiment = "Negative"
17     else:
18         sentiment = "Neutral"
19
20     return f"{sentiment} \n ({positive_count} positive words
    and {negative_count} negative words)"
```

Sentiment analysis: a minimal example

```
1 sentences = [  
2     "This movie was great and excellent!",  
3     "The food was terrible and awful.",  
4     "Today is a good day, but the weather is bad.",  
5     "I have nothing particular to say about this."  
6 ]  
7  
8 for sentence in sentences:  
9     sentiment = analyze_sentiment(sentence)  
10    print(f"Text: '{sentence}'")  
11    print(f"Sentiment: {sentiment}")  
12    print()  
13  
14 # Output:   Text: 'This movie was great and excellent!'  
15 #           Sentiment: Positive  
16 #           (2 positive words and 0 negative words)  
17 #  
18 #           Text: 'The food was terrible and awful.'  
19 #           Sentiment: Negative  
20 #           (0 positive words and 2 negative words)  
21
```


Dates can sometimes be a bit of a headache. Usual steps in working with dates

1. Transform string to date
2. Extract: year, month, day
3. Convert to seasons (winter, spring, summer, autumn)

Working with dates - string to date

```
1 data = {  
2     "id": [1, 2, 3, 4, 5],  
3     "date": [  
4         "Jan 1, 2024",  
5         "Feb 29, 2024",  
6         "Mar 15, 2023",  
7         "Dec 31, 2023",  
8         "Sep 5, 2024",  
9     ],  
10    "value": [100, 150, 200, 120, 180],  
11    }  
12  
13 df = pd.DataFrame(data)  
14  
15 # Convert the 'date' column to datetime type  
16 df["date"] = pd.to_datetime(df["date"], format="%b %d, %Y")  
17
```

Working with dates - extract year, month, day

```
1  # Extract year, month, and day into separate columns
2  df["year"] = df["date"].dt.year
3  df["month"] = df["date"].dt.month
4  df["day"] = df["date"].dt.day
```

```
5
6  print(df)
```

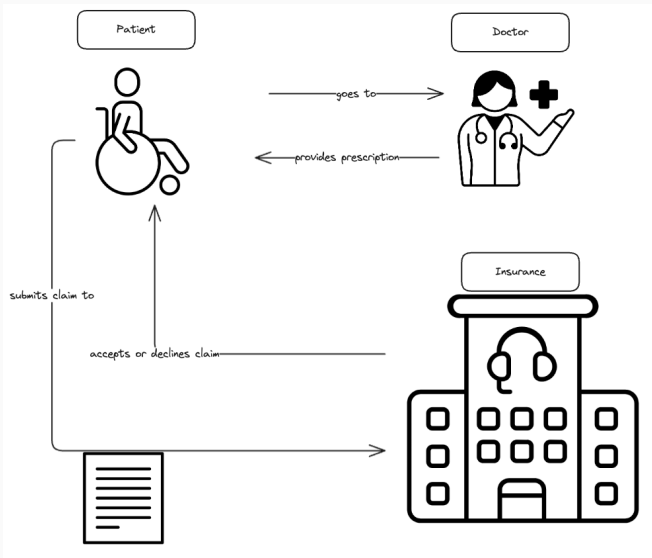
```
7  # Output:
```

```
8  #      id      date  value  year  month  day
9  # 0     1 2024-01-01    100  2024     1    1
10 # 1     2 2024-02-29    150  2024     2   29
11 # 2     3 2023-03-15    200  2023     3   15
12 # 3     4 2023-12-31    120  2023    12   31
13 # 4     5 2024-09-05    180  2024     9    5
```

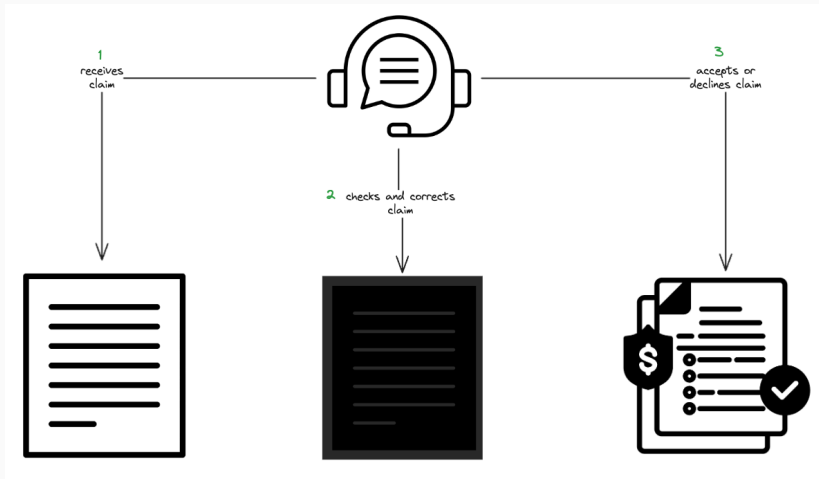
Working with dates - sub-setting to get seasons

```
1 def get_season(month):
2     if month in [12, 1, 2]:
3         return "Winter"
4     elif month in [3, 4, 5]:
5         return "Spring"
6     elif month in [6, 7, 8]:
7         return "Summer"
8     else:
9         return "Autumn"
10
11 # Add season column
12 df["season"] = df["month"].apply(get_season)
13
14 print(df)
15 # Output:
16 #      id      date  value  year  month  day  season
17 # 0     1 2024-01-01    100  2024     1    1  Winter
18 # 1     2 2024-02-29    150  2024     2   29  Winter
19 # 2     3 2023-03-15    200  2023     3   15  Spring
20 # 3     4 2023-12-31    120  2023    12   31  Winter
21 # 4     5 2024-09-05    180  2024     9    5  Autumn
```

Case study: Sometimes we also have to process the target variable



Three stages in handling an insurance claim



The data quickly gets complicated

claim_id	timestamp	age	obj_id	prescription	tfp	price	decision
1	09:27:43 2024-05-05	80	1	yes	2	100	
1	13:30:34 2024-05-05	80	1	yes	3	100	
1	16:25:30 2024-05-06	80	1	yes	3	100	1

...and even more complicated

claim_id	timestamp	age	obj_id	prescription	tfp	price	decision
2	10:22:22 2024-05-05	25	1			4	100
2	10:22:22 2024-05-05	25	2	yes		4	5000
2	13:12:12 2024-05-06	25	1	yes		4	100
2	13:12:12 2024-05-06	25	2	yes		4	5000
2	15:09:10 2024-05-07	25	1	yes		4	50
2	15:09:10 2024-05-07	25	2	yes		4	5000

So what is your label?

- We want to predict when we receive the insurance claim
- But the label we observe is only created after the claims handlers have worked on the claim

So what is your label?

- We want to predict when we receive the insurance claim
- But the label we observe is only created after the claims handlers have worked on the claim
- So we need to change the label from "accept/decline" to "accept without intervention/declined"

Merging tables

Bringing data together

- Your data will never come from a single table
- You can combine tables in several different ways
- Things can go wrong!

Several ways to merge your data

There are four main merging strategies:

1. **inner:** Keep keys that are present in both tables (intersection)
2. **left:** Keep keys in the left table and set non-matching values in the right table to NaN
3. **right:** Keep keys in the right table and set non-matching values in the right table to NaN
4. **outer:** Keep the union of keys found in both right and left tables (full join)

Example - four ways to merge these data frames

employee_id	employee_name
1	Alice
2	Diego
3	Aisha

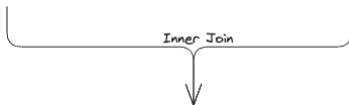
employee_id	dempartment_name
1	Engineer
2	HR
4	Marketing

Example - inner join

```
pd.merge(employees, departments, on='employee_id', how='inner')
```

employee_id	employee_name
1	Alice
2	Diego
3	Aisha

employee_id	dempartment_name
1	Engineer
2	HR
4	Marketing



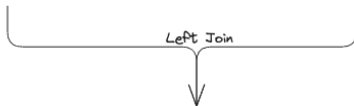
employee_id	employee_name	dempartment_name
1	Alice	Engineer
2	Diego	HR

Example - left join

```
pd.merge(employees, departments, on='employee_id', how='left')
```

employee_id	employee_name
1	Alice
2	Diego
3	Aisha

employee_id	dempartment_name
1	Engineer
2	HR
4	Marketing



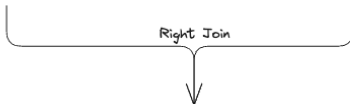
employee_id	employee_name	dempartment_name
1	Alice	Engineer
2	Diego	HR
3	Aisha	NaN

Example - right join

```
pd.merge(employees, departments, on='employee_id', how='right')
```

employee_id	employee_name
1	Alice
2	Diego
3	Aisha

employee_id	dempartment_name
1	Engineer
2	HR
4	Marketing



employee_id	employee_name	dempartment_name
1	Alice	Engineer
2	Diego	HR
4	NaN	Marketing

Example - outer join

```
pd.merge(employees, departments, on='employee_id', how='outer')
```

employee_id	employee_name
1	Alice
2	Diego
3	Aisha

employee_id	dempartment_name
1	Engineer
2	HR
4	Marketing

Outer Join

employee_id	employee_name	dempartment_name
1	Alice	Engineer
2	Diego	HR
3	Aisha	NaN
4	NaN	Marketing

Example - check your merge

```
pd.merge(employees, departments, on='employee_id', how='outer', indicator=True)
```

employee_id	employee_name
1	Alice
2	Diego
3	Aisha

employee_id	dempartment_name
1	Engineer
2	HR
4	Marketing

Outer Join

employee_id	employee_name	dempartment_name	_merge
1	Alice	Engineer	both
2	Diego	HR	both
3	Aisha	NaN	left_only
4	NaN	Marketing	right_only

Example - check your merge

```
outer_join["_merge"].value_counts()
```

_merge	
both	2
left_only	1
right_only	1

Some things to avoid merge mistakes

To avoid mistakes, try to:

Some things to avoid merge mistakes

To avoid mistakes, try to:

- Think about what you would expect before any merge

Some things to avoid merge mistakes

To avoid mistakes, try to:

- Think about what you would expect before any merge
- Calculate the number of rows (and columns) you expect after the merge
 - e.g. a left-join should always lead to the same number of rows as in the left-table before joining

Some things to avoid merge mistakes

To avoid mistakes, try to:

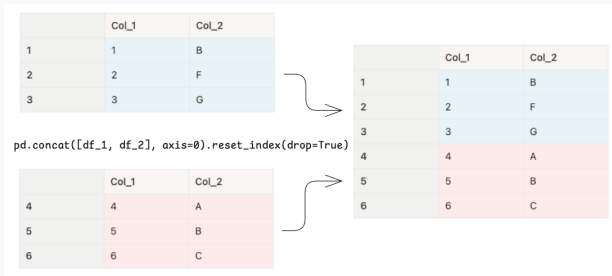
- Think about what you would expect before any merge
- Calculate the number of rows (and columns) you expect after the merge
 - e.g. a left-join should always lead to the same number of rows as in the left-table before joining
- Look at the merged table and do some quick checks (null values, the right values in the column, etc.)
- Use the indicator to check (left_only, right_only, both)
- `df["_merge"].value_counts()`

```
nlsy_combined['_merge'].value_counts()
```

```
_merge
both      241034
left_only    17
right_only    0
Name: count, dtype: int64
```

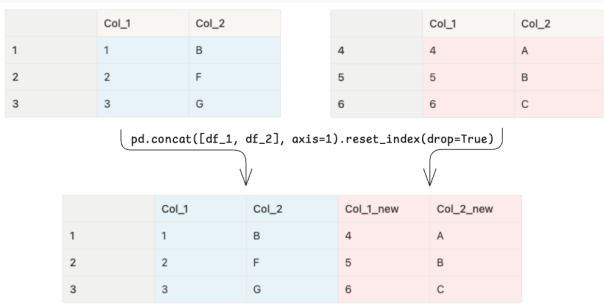
Figure 3: "Data Wrangling in Python"

Concatenating - pasting tables together row based



- Column names have to be exactly(!) the same
- Often used when new data of the same table is coming in

Concatenating - pasting tables together column based



Handling null values - missing data

Null values representation

There are different representations of missing data (knowing about this will hopefully help you avoid mistakes and unexpected bugs in the future):

- None: a Python object
- NaN (stands for: not a number): a floating-point value
- Null (more flexible; used by polars library)

What do you expect?

The representation of missing data sometimes tricks you and leads to unexpected bugs:

What do you expect when you use None

```
1 2 + None
```

```
2
```

compared to nan

```
1 import numpy as np
```

```
2
```

```
3 2 + float("nan") # python native
```

```
4 2 + np.nan      # numpy: usually used
```

```
5 2 * np.nan
```

```
6
```

What do you expect? Solution I

```
1 2 + None
```

```
2
```

```
TypeError                                Traceback (most recent call last)
File /Users/adrianochs/Desktop/data_wrangling.py:2
      1 # %%
----> 2 2 + None

TypeError: unsupported operand type(s) for +: 'int' and 'NoneType'
```

What do you expect? Solution II

```
1  import numpy as np
2
3  2 + float("nan")  # python native
4  2 + np.nan        # numpy: usually used
5  2 * np.nan
6  # Output: nan
7  # Output: nan
8  # Output: nan
9
```

- There is no error.
- This can sometimes lead to confusing results.

What do you expect?

What data type do you expect for this array?

```
1  vals1 = np.array([1, None, 3, 4])  
2  vals1  
3
```


What do you expect? Solution

```
1 vals1 = np.array([1, None, 3, 4])
2 vals1
3 # Output: array([1, None, 3, 4], dtype=object)
4
```

- The integer array has now an overall object data type
- Object is the general Python type and all operations are done in "pure Python"
- Numpy is built on C and is thus, quicker. However, it does not work on objects.

What do you expect?

What data type do you expect for this array?

```
1  vals2 = np.array([1, np.nan, 3, 4])  
2  vals2.dtype  
3
```

What do you expect? Solution

What data type do you expect for this array?

```
1  vals2 = np.array([1, np.nan, 3, 4])  
2  vals2.dtype  
3  # Output: dtype("float64")  
4
```

- The data type is upcasted to float.

The hidden bugs

Let's say you have an array like before:

```
1  vals2 = np.array([1, np.nan, 3, 4])
```

```
2
```

But you are not aware that there are nan values in it.

The hidden bugs

Let's say you have an array like before:

```
1     vals2 = np.array([1, np.nan, 3, 4])
```

```
2
```

But you are not aware that there are nan values in it.

(Maybe new data came in and usually, this column never has nans). Your program proceeds with:

```
1     vals2_mean = vals_2.mean()
```

```
2
```

The hidden bugs

Let's say you have an array like before:

```
1     vals2 = np.array([1, np.nan, 3, 4])
```

```
2
```

But you are not aware that there are nan values in it.

(Maybe new data came in and usually, this column never has nans). Your program proceeds with:

```
1     vals2_mean = vals_2.mean()
```

```
2
```

And this value is used in several calculations for your program or report:

```
1     report_1 = vals2_mean * number1
```

```
2     report_2 = vals2_mean * number2
```

```
3
```

What's the result?

Your report numbers are NaNs

You get the end results for your report numbers and all are nan without any errors.

```
import numpy as np

number1 = 8
vals2 = np.array([1, np.nan, 3, 4])
vals2_mean = vals2.mean()
report1 = vals2_mean * number1
print(report1)
```

[12] ✓ 0.0s

... nan

Another bug when you are not aware of different missing value representations

Let's use polars instead of pandas and count rows with missing values:

```
1 import polars as pl
2 import numpy as np
3
4 df = pl.DataFrame({'A': [1.2, 3.4, 4.3, np.nan]})
5 df.null_count()
6 # Output: 0
7
```

- NaN values are considered to be a type of floating point data and are not considered to be missing data in polars.
- Null values in polars indicate missing data and they can be used with all data types without upcasting them to floats.

From the Polars documentation

NaN values are considered to be a type of floating point data and are **not considered to be missing data** in Polars. This means:

- NaN values are **not** counted with the `null_count` method
- NaN values are filled when you use `fill_nan` method but are **not** filled with the `fill_null` method

Polars has `is_nan` and `fill_nan` methods which work in a similar way to the `is_null` and `fill_null` methods. The underlying Arrow arrays do not have a pre-computed validity bitmask for NaN values so this has to be computed for the `is_nan` method.

Figure 4: Polars documentation

Loading missing data from a csv

Sometimes missing values are "hidden" in a string:

```
1  NA_VALUES = ["NO CLUE", "N/A", "0"]
2
3  pd.read_csv(DATA_PATH, na_values=NA_VALUES)
4  pl.read_csv(DATA_PATH, null_values=NULL_VALUES)
5
```

How to deal with missing data?

Note

During data wrangling, make sure to represent missing values appropriately.

At feature engineering time, we will talk about **what to do** with missing data:

1. Drop data

- Drop the whole row (i.e. lose an observation)
- Drop the whole column (i.e. lose a whole feature)

2. Replace missing data

- Forward fill (i.e. fill with the value that came before it)
- Replace by mean or median
- More advanced imputation techniques

Save your data as Parquet file

Don't forget to save your wrangled data frame

```
1 df.to_parquet("cleaned_data.parquet") # Pandas
2 pl_df.write_parquet("pl_cleaned_data.parquet") # Polars
3
```

- After the wrangling, save your cleaned data frame
- This involves getting rid of irrelevant variables (no point in keeping those around)
- When you separate out your data wrangling from the next step of the DS pipeline, then you can just load the wrangled data next time and continue with your analysis directly
- Parquet format is specifically optimized for analytical workloads ([Medium article](#))
 - Columnar Storage: all values of a single column are stored together
 - Compression: supports various compression algorithms
 - → it's faster

Bonus

Git shortcuts

For those very familiar with Git, you can create shortcuts that add, commit, and push in one go to your global git config:

```
[user]
  name = Adrian Ochs
  email = example_email

[core]
  editor = $ubl
  excludesFile = ~/.gitignore

[alias]
  b = "If() { git branch $1 && git checkout $1 && git push origin --set-upstream $1; }; f"
  p = "If() { git add . && git commit -m \"$1\" && git push; }; f"
  c = "If() { git checkout $1 && git pull; }; f"
```

Figure 5: Adding shortcuts to your git config

Use Git Large File Storage (LFS)

- Git LFS (Large File Storage) is an extension of Git that helps to manage large files in a more efficient way.
- In standard Git, every file is stored in the .git repository, and every version of every file is tracked.
- Git LFS solves this by:
 - Storing large files outside of the main Git repository
 - In your Git repo, LFS replaces the large file with a lightweight pointer file. This pointer references the actual content stored elsewhere.
 - When you clone a repository that uses Git LFS, it only downloads the version of the large files that you need (e.g., the latest version), rather than every version that exists in the repo's history.

Git LFS setup

- Download it from [here](#)
- Then, navigate to your repo and:

```
1  git lfs install --global
2  git lfs track "*.jpg"
3  git add .gitattributes
4  git commit -m "Track large jpg with Git LFS"
5
```

NB: file tracking is repo-specific.

Going together over a data wrangling Kaggle notebook

Data wrangling notebook